

Lab 10: TPC-C Benchmark & Capacity Sizing Exercise (65 min)

The other half of Lab 8. Lab 8 asked "how fast can this schema go?" This lab asks "how much hardware do I buy to hit a number?"

Learning Objectives

By the end of this lab you will be able to:

- Run TPC-C at a chosen warehouse count and read the efficiency (`tpmC` vs theoretical max)
- Compare TPC-C, KV, and YCSB workloads and pick the right one for a sizing question
- Derive a node count from a throughput target using measured per-node capacity
- Apply the CPU / RAM / disk / network budgets per node and find which one binds first
- Observe admission control protecting the cluster under overload
- Produce a defensible sizing proposal for 1k, 10k, 100k, and 1M QPS targets

Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- At least 8 GB of free RAM (TPC-C at 10 warehouses on 3 nodes is comfortable)
- Lab 9's Prometheus/Grafana stack is useful here but not required

Setup

```
scripts/crdb up
mkdir -p /tmp/lab10           # the run logs you will read back go here
```

Two connection strings, and it matters which you use:

```
from your machine      postgresql://root@localhost:26257?sslmode=disable
from inside the cluster postgresql://root@crdb1:26257?sslmode=disable
```

`scripts/crdb run ...` executes inside node 1's container, so it uses the second form.

`--cache=.25 --max-sql-memory=.25` is the production-recommended split for a node that owns its machine. The lab nodes run with `.10 / .10` instead (see [docker/labs.yml](#)): three of them share one 8 GB Docker host, and each one sizes those percentages from the *host's* memory, not from a per-container limit — at `.25` the three nodes together could commit 11 GB and TPC-C at 30 warehouses OOM-kills one of them. Under-sizing a shared host is the same lesson as Lab 16's cgroup warning, one layer down.

Tasks

Part A: The Workload Suite — Pick the Right Tool (10 min)

```
# There is no `workload list` subcommand — the generators are the subcommands
# of `init` and `run`, so ask for their help:
scripts/crdb run workload init --help
scripts/crdb run workload run --help
```

Workload	Shape	Use it to answer
<code>kv</code>	Single-table point reads/writes	Raw per-node throughput ceiling
<code>ycsb</code>	Configurable read/write mix (A–F)	Read/write ratio sensitivity
<code>tpcc</code>	9-table OLTP with joins, txn mix, consistency checks	"Can this cluster run my OLTP app?"
<code>movr</code>	Multi-region ride-share schema	Multi-region latency behaviour
<code>bank</code>	Simple transfers between accounts	Contention and retry behaviour
<code>tpch</code>	Read-only analytical queries	Scan-heavy / OLAP-style load
<code>ttlbench</code>	Row-level TTL throughput	Cost of a TTL job on a big table

The generator list is version-specific — `cockroach workload run --help` on **your** version is the authority. (`ledger`, referenced in older material, is not present in v23.2.)

The trap: `kv` numbers look glorious and mean almost nothing for an application with joins, secondary indexes, and foreign keys. Use `kv` to find a ceiling; use `tpcc` to find what you'll actually get.

Part B: Run TPC-C (20 min)

TPC-C measures **tpmC** — new-order transactions per minute. Each warehouse has a theoretical max of **12.86 tpmC**, so efficiency = `actual_tpmC / (warehouses × 12.86)`. Anything above ~85% is a passing run; below that, the cluster is the bottleneck.

1. Load the dataset:

```
time scripts/crdb run workload fixtures import tpcc --warehouses=10 'postgresql://
root@crdb1:26257?sslmode=disable'
```

`fixtures import` uses `IMPORT INTO` under the hood — the same mechanism you measured in Lab 8 Part A. If it is unavailable offline, use `scripts/crdb run workload init tpcc --warehouses=10 'postgresql://root@crdb1:26257?sslmode=disable'`, which is slower because it inserts through the SQL layer.

2. **Look at what TPC-C's schema does right** — this is a schema-design lesson disguised as a benchmark:

```
SHOW CREATE TABLE tpcc.order_line;
SHOW CREATE TABLE tpcc.stock;
```

Note that every table leads its primary key with the warehouse id (`ol_w_id`, `s_w_id`). That is the **Per-Tenant Co-located PK** pattern (Playbook #3): one warehouse's rows sit together, so a new-order transaction touches one range instead of nine.

3. **Warm up, then measure:**

```
scripts/crdb run workload run tpcc --warehouses=10 --ramp=30s --duration=3m \
--display-every=15s 'postgresql://root@crdb1:26257?sslmode=disable' | tee /tmp/lab10/
tpcc-10.log
```

4. **Read the output.** The final block reports `tpmC`, efficiency, and per-transaction-type latency:

_elapsed	tpmC	efc	avg(ms)	p50(ms)	p90(ms)	p95(ms)	p99(ms)	pMax(ms)
180.0s	126.3	98.2%	12.4	11.0	21.0	26.2	41.9	71.3

Column	Meaning
tpmC	New-order transactions per minute
efc	Efficiency vs the 12.86/warehouse theoretical max
p99	Tail latency — the number your users feel

5. **Push until it breaks.** In principle you raise warehouses until efficiency drops below 85%; on this host something else breaks first (see the note after the table):

```
for W in 15 20; do      # see the note below before going higher
  scripts/crdb sql -e "DROP DATABASE IF EXISTS tpcc CASCADE;"
  # a fixture only imports into an empty schema
  scripts/crdb run workload fixtures import tpcc --warehouses=$W 'postgresql://
root@crdb1:26257?sslmode=disable'
  scripts/crdb run workload run tpcc --warehouses=$W --ramp=30s --duration=2m
    'postgresql://root@crdb1:26257?sslmode=disable' \
    | tail -3 | tee -a /tmp/lab10/tpcc-sweep.log
done
```

Warehouses	tpmC	Efficiency	p99 (ms)	Bottleneck (CPU / disk / contention?)
10				
15				
20				

Why the sweep stops at 20 on this host. Measured on an 8 GB Docker host: 20 warehouses imports and runs at ~100% efficiency with each node at ~1.5 GB; the **30** warehouse import OOM-kills a node partway through (`docker events --filter event=oom` shows it) — the ingest path needs memory beyond `--cache` and `--max-sql-memory`, and three nodes share the machine. You will not see efficiency fall below 85% here; you hit the *other* ceiling first. That is a real capacity result: on this hardware the binding constraint is memory during bulk ingest, not CPU during steady state — write it in the bottleneck column.

6. Identify the bottleneck at the failing point while a run is in flight:

```
-- Node-level: CPU and admission control
SELECT node_id,
       metrics->'sys.cpu.combined.percent-normalized' AS cpu,
       metrics->'admission.wait_durations.kv-p99'      AS adm_wait_p99
FROM crdb_internal.kv_node_status;

-- Store-level: the storage engine
SELECT node_id, store_id,
       metrics->'rocksdb.read-amplification' AS read_amp,
       metrics->'capacity.available'         AS capacity_available
FROM crdb_internal.kv_store_status;
```

Node metrics vs store metrics. `crdb_internal.kv_node_status` carries process-level metrics (`sys.*`, `sql.*`, `admission.*`). Disk and storage-engine metrics — `capacity.*`, `rocksdb.*`, `queue.*` — live in `crdb_internal.kv_store_status`, one row per store. Asking the wrong view returns `NULL` rather than an error, so the mistake is silent.

```
SELECT count(*) AS contention_events
FROM crdb_internal.transaction_contention_events
WHERE collection_ts > now() - INTERVAL '2 minutes';
```

Part C: Per-Node Capacity — Measure, Then Extrapolate (15 min)

The published Cockroach Labs guidance is a starting point; your measured number is the one you defend in a design review.

Reference budgets per node (production hardware, not a laptop):

Resource	Guidance	Why
CPU	4–16 vCPU; ≥ 8 for production	Below 4 vCPU, background work starves foreground SQL
RAM	4 GB per vCPU	Cache (25%) + SQL memory (25%) + Go heap + OS page cache
Disk	≤ 2.5 TB per node (up to 10 TB with large nodes and testing)	Recovery and rebalance time scales with store size
Disk IOPS	≥ 500 IOPS and ≥ 30 MB/s per vCPU	Raft log + LSM compaction are write-amplifying
Network	≥ 1 Gb/s; multi-region: latency matters more than bandwidth	Every write is a Raft round trip
Store/CPU ratio	~150 GB per vCPU write-heavy; ~320 GB per vCPU read-heavy	Compaction cost scales with data per core

1. Measure your laptop's per-node ceiling with kv :

```
scripts/crdb run workload init kv --drop 'postgresql://root@crdb1:26257?sslmode=disable'
scripts/crdb run workload run kv --duration=60s --concurrency=64 --read-percent=95
    'postgresql://root@crdb1:26257?sslmode=disable' | tail -3
scripts/crdb run workload run kv --duration=60s --concurrency=64 --read-percent=50
    'postgresql://root@crdb1:26257?sslmode=disable' | tail -3
scripts/crdb run workload run kv --duration=60s --concurrency=64 --read-percent=0
    'postgresql://root@crdb1:26257?sslmode=disable' | tail -3
```

Read %	ops/sec	p99 (ms)	ops/sec per node
95 (read-heavy)			
50 (mixed)			
0 (write-only)			

2. The extrapolation, stated honestly:

```
nodes_needed = ceil( target_QPS / (measured_ops_per_node × utilization_target) )
```

with `utilization_target = 0.5` — you size for half the measured ceiling so the cluster still serves traffic during a node failure, a rebalance, and a rolling upgrade.

3. Why writes cost more than reads. A write with `replication factor = 3` is:

- 1 leaseholder write + 2 follower writes = 3 disk writes
- 1 Raft round trip (quorum = 2 of 3) before the client sees success
- plus 1 write per secondary index, each with the same 3× replication

So a table with two secondary indexes turns one logical write into **nine** physical writes. That factor is the single most useful number in a capacity conversation, and it is decided by your **schema**, not your hardware.

Part E: Sizing Exercises — 1k / 10k / 100k / 1M QPS (20 min)

Work these in pairs. Use your measured numbers from Part C, the reference budgets from the table above, and state every assumption. There is no single right answer; there is a defensible answer.

Common assumptions to state up front:

- Read/write mix
- Average row size and rows per transaction
- Replication factor (3 unless stated)
- Number of secondary indexes (write amplification!)
- Retention / total dataset size
- Utilization target (0.5 for production)
- Survival goal (zone vs region)

Scenario 1 — 1,000 QPS, 90% reads, 200 GB, single region

- Write QPS: 100/s → ×3 replication → 300 physical writes/s
- Per-node ceiling (measured): _____ ops/s
- Nodes for throughput: _____
- Nodes for storage: 200 GB ÷ 2.5 TB per node = 1 → replication ×3 = 600 GB total
- Nodes for survival: minimum 3 for **ZONE** survival
- **Answer:** _____ nodes × _____ vCPU × _____ GB RAM × _____ disk

Scenario 2 — 10,000 QPS, 70% reads, 2 TB, single region, 2 secondary indexes

- Write QPS: 3,000/s → ×3 replication × 3 KV writes (base + 2 indexes) = _____ physical writes/s
- Storage: 2 TB × 3 = 6 TB → at 2.5 TB/node that's _____ nodes minimum for storage alone
- Store/CPU ratio: at 150 GB per vCPU write-heavy, 6 TB needs _____ vCPU total
- **Which constraint binds first — CPU, disk capacity, or IOPS?**

Scenario 3 — 100,000 QPS, 50/50, 20 TB, 3 regions, **REGION** survival

- **REGION** survival needs ≥ 3 regions and ≥ 5 replicas — recompute the replication factor
- Cross-region write latency: quorum spans regions, so writes pay one inter-region RTT (~60–70 ms us-east ↔ us-west). What does that do to your per-connection throughput?
- Which tables should be **REGIONAL BY ROW**, which **GLOBAL**, which **REGIONAL BY TABLE**?
- **Answer:** _____ nodes per region × 3 regions

Scenario 4 — 1,000,000 QPS, 95% reads, 100 TB, global

- At this scale the answer is a *topology*, not a node count. Sketch:
 - Which reads can be served by follower reads / **GLOBAL** tables?
 - What is the schema partitioning key, and does it avoid a global hotspot?
 - How do you keep any single range from becoming the bottleneck?
- **The real question:** what in the schema would stop this from scaling, and how do you find it before you buy the hardware? (Answer: Lab 8's PK bake-off and Lab 6's plan reading.)

Deliverable

Each pair presents one scenario: node count, instance shape, the binding constraint, and the one schema decision that most changes the answer.

Optional — If Time Allows

These parts are not required to complete the lab; they extend it by about 10 minutes. Do them if you finish early, or after class — the cluster and data from the core parts carry over.

Part D: Admission Control Under Overload (10 min)

1. **Confirm admission control is on** (default in modern versions):

```
SHOW CLUSTER SETTING admission.kv.enabled;  
SHOW CLUSTER SETTING admission.sql_kv_response.enabled;  
SHOW CLUSTER SETTING admission.sql_sql_response.enabled;
```

2. **Overload the cluster deliberately** — a big analytical scan alongside the OLTP workload:

```
scripts/crdb run workload run tpcc --warehouses=10 --duration=3m 'postgresql://  
root@crdb1:26257?sslmode=disable' > /tmp/lab10/oltp.log 2>&1 &  
  
for i in $(seq 1 8); do  
  scripts/crdb sql -e "  
    SELECT count(*), sum(ol_amount) FROM tpcc.order_line;  
    SELECT count(*) FROM tpcc.stock a JOIN tpcc.stock b ON a.s_i_id = b.s_i_id LIMIT 1;" &  
done  
wait
```

3. **Watch admission control queue the low-priority work:**

```
SELECT node_id,  
  metrics->>'admission.wait_durations.kv-p99' AS kv_wait_p99,  
  metrics->>'admission.wait_queue_length.kv' AS kv_queue,  
  metrics->>'admission.granter.io_tokens_exhausted_duration.kv' AS io_exhausted  
FROM crdb_internal.kv_node_status;
```

4. **Compare the OLTP p99 during overload** to the clean run in Part B. Admission control's job is to make the *background* work wait so the *foreground* work degrades gracefully instead of collapsing.
5. **Protect a tenant/workload explicitly:**

```
-- Tag the analytical connection so its work is deprioritized
SET application_name = 'analytics';
SET default_transaction_quality_of_service = 'background';

-- And the OLTP one
SET default_transaction_quality_of_service = 'critical';
```

Re-run step 2 with the QoS settings applied and compare the OLTP p99.

Cleanup

```
scripts/crdb down
```

Lab 10 Deliverables

□ **TPC-C run** at 10 warehouses with efficiency $\geq 85\%$, numbers recorded □ **Sweep** to the point where efficiency drops, with the bottleneck identified □ **Per-node ceiling** measured at three read/write mixes □ **Write amplification** computed for a schema with secondary indexes □ **Admission control** observed protecting foreground work under overload □ **Four sizing proposals** with stated assumptions and a named binding constraint

Challenge Exercises

1. **Efficiency vs node count.** Re-run TPC-C at your failing warehouse count on a 5-node cluster. Did efficiency recover linearly? If not, what didn't scale?
2. **The index tax, in tpmC.** Add two secondary indexes to `tpcc.order_line` and re-run. Convert the tpmC drop into a "cost per index" number you could quote in a code review.
3. **YCSB workload sensitivity.** Run `ycsb` variants A (50/50), B (95/5), and C (100% read) at fixed concurrency. Plot ops/s against read percentage. Where is the curve steepest, and why?
4. **Size for a real app.** Take an application you actually run, write down its assumptions using the list in Part E, and produce a node count. Then argue against your own answer.

Reference

Command	Purpose
<code>cockroach workload init --help</code>	Show available workload generators
<code>cockroach workload fixtures import tpcc --warehouses=N</code>	Fast TPC-C load via IMPORT
<code>cockroach workload run tpcc --ramp=30s --duration=3m</code>	Measured TPC-C run

Command	Purpose
<code>cockroach workload run kv --read-percent=P --concurrency=C</code>	Per-node ceiling
<code>crdb_internal.kv_node_status</code>	CPU, read-amp, admission metrics per node
<code>SET default_transaction_quality_of_service = 'background'</code>	Deprioritize a workload
<code>SHOW CLUSTER SETTING admission.kv.enabled</code>	Admission control state